

Schema design strategies and scaling solution in MongoDB - Armeanca Elis-Daniel, 342

1. Technical documentation:
 - a. Software and Environment
2. Demonstrate Data Modeling and Collection Organization
 - a. Define entities and relationship:
 - i. Entități
 - ii. Relații
3. Compare embedding vs referencing strategies and justify schema choice & implement at least 2 different schema models and discuss pros/cons
 - a. Embedding vs Referencing. Schema Models
 - i. Model 1 – Referencing
 - ii. Model 2 – Embedding
 - iii. Concluzie
4. Implement CRUD Operations
 - a. Implement Create, Read, Update, Delete with realistic queries & include aggregation pipelines for more complex queries
 - i. CRUD Operations
 - ii. Aggregation Pipelines
5. Apply Scaling & Optimization strategies
 - a. Apply indexes and evaluate query performance (before/after).
6. REPLICATION AND READ/WRITE FAILOVER
 - a. Pasi efectuați:
 - b. Rezultat:
7. SHARDING
 - a. Test performanță: targeted vs scatter-gather
8. VERTICAL & HORIZONTAL SCALING
 - a. Vertical scaling
 - b. Horizontal scaling
 - c. Concluzie
9. Common pitfalls in schema design and CRUD operations

11. Cand MongoDB este mai potrivit decat SQL pentru acest dataset

Technical documentation:

Am folosit MongoDB si Mongoose.

Software and Environment

- OS: Windows 10
- Database: MongoDB
- Tools: MongoDB Compass, mongosh
- Containerization: Docker
- Programming: JavaScript (Mongoose)
- VM usage: none

Demonstrate Data Modeling and Collection Organization

Define entities and relationship:

Domeniul ales este: Gestiunea organizarii de evenimente de tip conferinta.

Diagrama ERD:

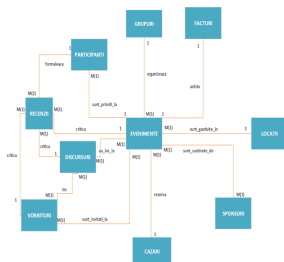
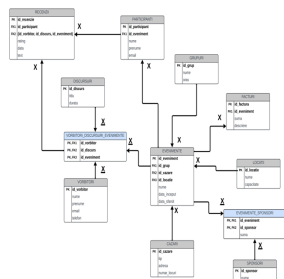


Diagrama conceptuala:



Entități

- Grupuri
- Evenimente
- Locații

- Participanți
- Vorbitori
- Discursuri
- Recenzii
- Cazări
- Sponsori
- Facturi
- Vorbitori_Discursuri_Evenimente (entitate de legătură)
- Evenimente_Sponsori (entitate de legătură)

Relații

- Grupuri **1** → **N** Evenimente
- Evenimente **N** → **1** Locații
- Evenimente **1** → **N** Participanți
- Evenimente **1** → **N** Discursuri
- Vorbitori **N** → **N** Discursuri (prin Vorbitori_Discursuri_Evenimente)
- Vorbitori **N** → **N** Evenimente (prin Vorbitori_Discursuri_Evenimente)
- Participanți **1** → **N** Recenzii
- Discursuri **1** → **N** Recenzii
- Vorbitori **1** → **N** Recenzii
- Evenimente **1** → **1** Cazări
- Evenimente **1** → **N** Facturi
- Evenimente **N** → **N** Sponsori (prin Evenimente_Sponsori)

Baza de date conține informații referitoare la gestiunea organizării unor evenimente de tip conferință.

Organizarea evenimentelor se realizează astfel:

Un eveniment este găzduit de un grup, care îl planifică și coordonează. Astfel că evenimentul se desfășoară într-o locație, achită facturi, este ajutat de sponsori, rezervă cazări, primește participanți.

În cadrul evenimentului propriu-zis au loc mai multe discursuri, care au mai multe schițe generale centralizate, pe care fiecare vorbitor le adaptează la stilul propriu. Astfel, elementele principale ale evenimentelor sunt vorbitorii și discursurile pe care aceștia le țin. Evenimentul asigură cazarea vorbitorilor, iar participanții pot formula recenzii despre fiecare discurs în parte.

Prin urmare, baza de date îmbină atât partea „din spatele cortinei”, cât și ceea ce are loc propriu-zis „pe scenă”.

Pe baza acestui model conceptual și a relațiilor identificate, în etapele următoare au fost analizate și implementate diferite strategii de modelare a datelor în MongoDB, folosind

embedding și referencing.

Compare embedding vs referencing strategies and justify schema choice & implement at least 2 different schema models and discuss pros/cons

Embedding vs Referencing. Schema Models

În cadrul proiectului au fost analizate și implementate două strategii de modelare a datelor în MongoDB: **referencing** și **embedding**.

Modelul bazat pe **referencing** (model normalizat) utilizează colecții separate pentru fiecare entitate, iar relațiile dintre acestea sunt realizate prin referințe de tip *ObjectId*. În acest proiect, **strategia dominantă este referencing**, fiind utilizată pentru majoritatea entităților, precum: **discursuri, vorbitori, participanți, sponsori, recenzii**, precum și pentru relațiile de tip M:N, implementate prin colecții de legătură (*evenimente_sponsori, vorbitori_discursuri_evenimente*).

Modelul bazat pe **embedding** (model denormalizat) include datele frecvent accesate împreună direct în documentul principal. În cadrul proiectului, **entitatea evenimente a fost implementată și într-o formă embedded**, în colecția *evenimente_embedded*, care conține **snapshot-uri** ale informațiilor asociate unui eveniment (grup, locație, cazare, sponsori, facturi, programul discursurilor și statistici agregate). Această colecție nu înlocuiește modelul bazat pe referencing, ci este utilizată **ca alternativă**, pentru a evidenția avantajele și dezavantajele denormalizării.

Model 1 – Referencing

Avantaje:

- structură flexibilă și scalabilă
- documente de dimensiuni reduse
- actualizări independente ale entităților
- potrivit pentru relații complexe și interogări bazate pe agregări

Dezavantaje:

- necesită mai multe interogări pentru obținerea tuturor datelor
- pot fi necesare agregări (\$lookup, \$group) pentru corelarea informațiilor

Model 2 – Embedding

Avantaje:

- acces rapid la toate informațiile relevante ale unui eveniment
- număr redus de interogări

Dezavantaje:

- documente de dimensiuni mai mari
- redundanță de date
- actualizări mai costisitoare în cazul modificărilor frecvente

Concluzie

Modelul principal utilizat în proiect este **cel bazat pe referencing**, deoarece entitățile sunt independente, se modifică separat și permit o scalabilitate mai bună. Modelul **embedded a fost implementat complementar**, ca mecanism de **denormalizare și comparație**, pentru a evidenția trade-off-urile dintre **flexibilitate, consistență și performanță** în MongoDB.

MongoDB nu impune o schemă fixă în mod implicit. Structura documentelor este definită la momentul inserării, iar o colecție poate conține documente cu structuri diferite, dacă nu este definit explicit un validator de schemă, ceea ce oferă flexibilitate în alegerea și combinarea strategiilor de modelare.

colecția Evenimente referenced:

```
Atlas atlas-1168lo-shard-0 [primary] gestiune_conferinte> db.evenimente.findOne()
{
  _id: ObjectId('694e7ee94cc153765a1e2624'),
  id_grup: ObjectId('694e7e514cc153765a1e2621'),
  id_locatie: ObjectId('694e7ea24cc153765a1e2622'),
  id_cazare: ObjectId('694e7edc4cc153765a1e2623'),
  nume: 'Conferinta AI 2020',
  data_inceput: ISODate('2026-03-10T00:00:00.000Z'),
  data_sfarsit: ISODate('2026-03-12T00:00:00.000Z')
}
```

colecția Evenimente_embedded:

```
Atlas atlas-1168lo-shard-0 [primary] gestiune_conferinte> db.evenimente_embedded.findOne()
{
  _id: ObjectId('694e81104cc153765a1e262c'),
  ref_eventiment_id: ObjectId('694e7ee94cc153765a1e2624'),
  nume: 'Conferinta AI 2020',
  data_inceput: ISODate('2026-03-10T00:00:00.000Z'),
  data_sfarsit: ISODate('2026-03-12T00:00:00.000Z'),
  grup: {
    id_grup: ObjectId('694e7e514cc153765a1e2621'),
    nume: 'Bucurestii Hard',
    oras: 'Bucuresti'
  },
  locatie: {
    id_locatie: ObjectId('694e7ea24cc153765a1e2622'),
    nume: 'Aula Magna',
    capacitate: 300
  },
  cazare: {
    id_cazare: ObjectId('694e7edc4cc153765a1e2623'),
    tip: 'Hotel',
    adresa: 'Bulevardul Gh. Magheru',
    numar_locuri: 120
  },
  facturi: [ { suma: 1200, descriere: 'Inchiriere sala' } ],
  sponsori: [
    {
      id_sponsor: ObjectId('694e7ef04cc153765a1e2625'),
      nume: 'TechCorp'
    }
  ],
  program: [
    {
      id_vorbitor: ObjectId('694e7f284cc153765a1e2627'),
      nume: 'Popescu',
      prenume: 'Andrei',
      email: 'andrei@popescu.com',
      id_discurs: ObjectId('694e7f324cc153765a1e2628'),
      titlu: 'Vectori în 101',
      durata: 45
    }
  ],
  stats: { nr_participanti: 1, nr_recenzii: 1, rating_mediu: 9 }
}
```

toate colecțiile, cu excepția evenimente_embedded sunt referenced:

```
Atlas atlas-1168lo-shard-0 [primary] gestiune_conferinte> show collections
cazari
discursuri
evenimente
evenimente_embedded
evenimente_sponsori
facturi
grupuri
locatii
participanti
recenzii
sponsori
vorbitori
vorbitori_discursuri_evenimente
```

Implement CRUD Operations

Implement Create, Read, Update, Delete with realistic queries & include aggregation pipelines for more complex queries

CRUD Operations

Am implementat operatiile CRUD printr-o serie de scripturi rulate pe baza de date *gestiune_conferinte*. Conectarea la baza se face folosind Mongoose pentru connection management, iar operatiile sunt executate direct pe colectii.

- **Create:** inserare documente noi in colectiile de baza si in colectia *evenimente*.
- **Read:** verificarea rezultatelor dupa inserare si dupa modificari prin interogari in Compass/mongosh (ex: *find*, *countDocuments*).
- **Update:** modificarea documentelor existente (ex: update nume eveniment / adaugare legaturi).
- **Delete:** stergerea documentelor dupa criterii (ex: dupa nume sau id).
- **Error handling / realism:** folosesc date reale pentru domeniul conferintelor si verific rezultatele prin numar de documente si continutul documentelor.

Aggregation Pipelines

Am implementat o interogare complexa folosind aggregation pipeline pentru a combina date din mai multe colectii. Pipeline-ul porneste din *evenimente* si foloseste *\$lookup* pentru a aduce informatii despre grupul asociat fiecarui eveniment, apoi *\$unwind* si *\$project* pentru un rezultat final compact (nume eveniment + nume grup).

Urmatoarele script-uri din proiect ilustreaza operatiile READ:

01_insert_core.mongoose.js

```
C:\FACULTATE\An 3\BD\bdnv-project>npm run mongo:01

> bdnv-project@1.0.0 mongo:01
> node scripts/01_insert_core.mongoose.js

(node:21420) [MODULE_TYPELESS_PACKAGE_JSON] Warning:
t_core.mongoose.js is not specified and it doesn't p
Reparsing as ES module because module syntax was det
To eliminate this warning, add "type": "module" to C
(Use `node --trace-warnings ...` to show where the w
Connected to MongoDB
=== INSERT CORE DONE ===
grupuri: 3, locatii: 3, cazari: 3
sponsori: 3, vorbitori: 3, discursuri: 3
```

02_insert_evenimente_ref.mongoose.js

```
C:\FACULTATE\An 3\BD\bdnv-project>npm run mongo:02

> bdnv-project@1.0.0 mongo:02
> node scripts/02_insert_evenimente_ref.mongoose.js

(node:15164) [MODULE_TYPELESS_PACKAGE_JSON] Warning: Module
t_evenimente_ref.mongoose.js is not specified and it doesn't
Reparsing as ES module because module syntax was detected. T
To eliminate this warning, add "type": "module" to C:\FACULT
(Use `node --trace-warnings ...` to show where the warning w
Connected to MongoDB
=== INSERT EVENIMENTE REF DONE === 1
```

03_insert_links.mongoose.js

```
C:\FACULTATE\An 3\BD\bdnv-project>npm run mongo:03

> bdnv-project@1.0.0 mongo:03
> node scripts/03_insert_links.mongoose.js

(node:28300) [MODULE_TYPELESS_PACKAGE_JSON] Warning: Module
t_links.mongoose.js is not specified and it doesn't parse as
Reparsing as ES module because module syntax was detected. T
To eliminate this warning, add "type": "module" to C:\FACULT
(Use `node --trace-warnings ...` to show where the warning w
Connected to MongoDB
=== INSERT LINKS DONE ===
```

04_update.mongoose.js

```
C:\FACULTATE\An 3\BD\bdnv-project>npm run mongo:04

> bdnv-project@1.0.0 mongo:04
> node scripts/04_update.mongoose.js

(node:20524) [MODULE_TYPELESS_PACKAGE_JSON] Warning: Module
e.mongoose.js is not specified and it doesn't parse as Commo
Reparsing as ES module because module syntax was detected. T
To eliminate this warning, add "type": "module" to C:\FACULT
(Use `node --trace-warnings ...` to show where the warning w
Connected to MongoDB
UPDATE -> matched: 1 modified: 1
```

05_delete.mongoose.js

```

C:\FACULTATE\An 3\BD\bdnv-project>npm run mongo:05

> bdnv-project@1.0.0 mongo:05
> node scripts/05_delete.mongoose.js

(node:26060) [MODULE_TYPELESS_PACKAGE_JSON] Warning: Module
e.mongoose.js is not specified and it doesn't parse as CommonJS
Reparsing as ES module because module syntax was detected. To
eliminate this warning, add "type": "module" to C:\FACULTATE\An 3\BD\bdnv-project\package.json
(Use `node --trace-warnings ...` to show where the warning was created)
Connected to MongoDB
DELETE -> deleted: 1

```

READevenimente.mongoose.js

```

C:\FACULTATE\An 3\BD\bdnv-project>npm run mongo:read

> bdnv-project@1.0.0 mongo:read
> node scripts/READevenimente.mongoose.js

(node:4848) [MODULE_TYPELESS_PACKAGE_JSON] Warning: Module type of file://...
and it doesn't parse as CommonJS.
Reparsing as ES module because module syntax was detected. This incurs a performance penalty.
To eliminate this warning, add "type": "module" to C:\FACULTATE\An 3\BD\bdnv-project\package.json
(Use `node --trace-warnings ...` to show where the warning was created)
Connected to MongoDB
=== READ EVENIMENTE REF ===
Numar evenimente: 3
1. Conferinta AI 2026 | 2026-06-01 -> 2026-06-02
2. Conferinta Cloud 2026 | 2026-01-15 -> 2026-01-15
3. Conferinta Security 2026 | 2026-09-15 -> 2026-09-16

```

READmodel.mongoose.js (citeste evenimentele folosind modelul)


```

Numar evenimente: 3
[
  {
    _id: new ObjectId('6969487db2f2059cf70fc191'),
    nume: 'Conferinta AI 2026',
    data_inceput: 2026-06-01T00:00:00.000Z,
    data_sfarsit: 2026-06-02T00:00:00.000Z,
    id_grup: new ObjectId('694e7e514cc153765a1e2621'),
    id_locatie: new ObjectId('694e7ea24cc153765a1e2622'),
    id_cazare: new ObjectId('694e7edc4cc153765a1e2623')
  },
  {
    _id: new ObjectId('696949730cbb5778ce1e2621'),
    nume: 'Conferinta Cloud 2026',
    id_grup: new ObjectId('694e7e514cc153765a1e2621'),
    data_inceput: 2026-01-15T20:09:23.723Z,
    data_sfarsit: 2026-01-15T20:09:23.723Z
  },
  {
    _id: new ObjectId('69694a030cbb5778ce1e2623'),
    nume: 'Conferinta Security 2026',
    data_inceput: 2026-09-15T00:00:00.000Z,
    data_sfarsit: 2026-09-16T00:00:00.000Z,
    id_grup: new ObjectId('696949a30cbb5778ce1e2622')
  }
]

```

Acest script ilustreaza agregarea: 06_aggregations.mongoose.js

```

AGGREGATION RESULT: [
  {
    _id: new ObjectId('6969487db2f2059cf70fc191'),
    nume: 'Conferinta AI 2026',
    grup: { nume: 'Bucuresti Nord' }
  },
  {
    _id: new ObjectId('696949730cbb5778ce1e2621'),
    nume: 'Conferinta Cloud 2026',
    grup: { nume: 'Bucuresti Nord' }
  },
  {
    _id: new ObjectId('69694a030cbb5778ce1e2623'),
    nume: 'Conferinta Security 2026',
    grup: { nume: 'Bucuresti Sud' }
  }
]

```

Video demonstrativ:

https://youtu.be/uHLMZu_R6k

Apply Scaling & Optimization strategies

Apply indexes and evaluate query performance (before/after)

Colecția *recenzii* conține aproximativ **10.000 de documente**. Scopul acestui experiment a fost să demonstrăm impactul **indexării** asupra performanței query-urilor în MongoDB.

A fost ales câmpul *id_participant*, deoarece este **selectiv** (valoarea apare o singură dată în colecție), ceea ce permite evidențierea clară a diferenței dintre un query fără index și unul indexat.

Query BEFORE index:

```
db.recenzii.explain("executionStats").find({
  id_participant: ObjectId("6951780f003f8694041e2624")
})
```

Output:

```
executionStats: {
  executionSuccess: true,
  nReturned: 1,
  executionTimeMillis: 5,
  totalKeysExamined: 0,
  totalDocsExamined: 10001,
  executionStages: {
    isCached: false,
    stage: 'COLLSCAN',
    filter: {
      id_participant: { '$eq': ObjectId('6951780f003f8694041e2624') }
    },
    nReturned: 1,
    executionTimeMillisEstimate: 6,
    works: 10002,
    advanced: 1,
    needTime: 10000,
    needYield: 0,
    saveState: 0,
    restoreState: 0,
    isEOF: 1,
    direction: 'forward',
    docsExamined: 10001
  }
}
```

Partea relevantă: 10001 documente examinate, și COLLSCAN (scanare completă a colecției) - au fost examinate 10001 documente pentru un singur rezultat

SE CREEAZA UN INDEX:

```
db.recenzii.createIndex({ id_participant: 1 })
```

Acelasi QUERY AFTER INDEX:

```

executionStats: {
  executionSuccess: true,
  nReturned: 1,
  executionTimeMillis: 1,
  totalKeysExamined: 1,
  totalDocsExamined: 1,
  executionStages: {
    isCached: false,
    stage: 'FETCH',
    nReturned: 1,
    executionTimeMillisEstimate: 1,
    works: 2,
    advanced: 1,
    needTime: 0,
    needYield: 0,
    saveState: 0,
    restoreState: 0,
    isEOF: 1,
    docsExamined: 1,
    alreadyHasObj: 0,
    inputStage: {
      stage: 'IXSCAN',
      nReturned: 1,
      executionTimeMillisEstimate: 1,
      works: 2,
      advanced: 1,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      keyPattern: { id_participant: 1 },
      indexName: 'id_participant_1',
      isMultiKey: false,
      multiKeyPaths: { id_participant: [] },
      isUnique: false,
      isSparse: false,
      isPartial: false,
      indexVersion: 2,
      direction: 'forward',
      indexBounds: {

```

Partea relevantă: a fost examinat un singur document, numărul de documente scăzând de la 10001 la 1; timpul de execuție a fost redus semnificativ; IXSCAN folosit (scanare pe index)

Indexarea câmpurilor **selective** îmbunătățește semnificativ performanța query-urilor în MongoDB. În acest caz, utilizarea unui index pe *id_participant* a eliminat scanarea completă a colecției și a redus drastic costul interogării.

REPLICATION AND READ/WRITE FAILOVER

Rulam 3 noduri în Docker și le pornim:

```

docker compose up -d
docker ps

```

Initializam replica set:

```
docker exec -it mongo1 mongosh
rs.initiate({
  _id: "rs0",
  members: [
    { _id: 0, host: "mongo1:27017" },
    { _id: 1, host: "mongo2:27017" },
    { _id: 2, host: "mongo3:27017" }
  ]})
```

Verificam daca 1 PRIMARY si 2 SECONDARY:

```
rs.status().members.map(m => ({ name: m.name, stateStr: m.stateStr })))
```

iesim -> exit

executam nodul primary -> selectam baza de date -> insert un sponsor -> afisam documentele SPONSORI

```
docker exec -it mongo1 mongosh
use gestiune_conferinte
db.sponsori.insertOne({
  nume: "TechCorp",
  phase: "before_failover",
  ts: new Date()
})

db.sponsori.find().sort({ ts: 1 })

exit
```

Simulam caderea PRIMARY

```
docker stop mongo1
docker ps
```

Intram pe noul PRIMARY si verificam rolurile:

```
docker exec -it mongo2 mongosh
rs.status().members.map(m => ({ name: m.name, stateStr: m.stateStr })))
```

Insertam un nou sponsor dupa failover si verificam daca sunt ambele documente

```
use gestiune_conferinte

db.sponsori.insertOne({
  nume: "CloudSystems",
  phase: "after_failover",
  ts: new Date()
})

db.sponsori.find().sort({ ts: 1 })

exit
```

Pornim nodul initial si revine ca SECONDARY:

```
docker start mongo1
docker exec -it mongo2 mongosh
rs.status().members.map(m => ({ name: m.name, stateStr: m.stateStr })))
```

VIDEOUL pentru demonstratie:

 <https://youtu.be/0FJN7k1NpP0>

Rezumat: A fost configurat un replica set MongoDB cu 3 noduri (rs0), folosind Docker (mongo1, mongo2, mongo3). Colectia din proiect **sponsori** a fost folosita pentru a demonstra replicarea, folosind documente minimale.

Pasi efectuati:

1. Au fost pornite containerele si a fost initializat replica set-ul folosind rs.initiate().
2. A fost inserat un document in colectia conferinte.sponsori in timp ce nodul mongo1 avea rolul de PRIMARY (before_failover).
3. Nodul PRIMARY a fost oprit (docker stop mongo1) pentru a simula o cadere a serverului.
4. A fost verificata alegerea automata a unui nou nod PRIMARY folosind rs.status().
5. A fost inserat un al doilea document dupa failover pe noul nod PRIMARY (after_failover).
6. Nodul initial a fost repornit (docker start mongo1) si s-a confirmat ca a reintrat in replica set ca nod SECONDARY.

Rezultat:

- Datele au ramas disponibile si consistente dupa caderea nodului PRIMARY.

- Operatiile de scriere au continuat pe noul nod PRIMARY, demonstrand disponibilitatea ridicata oferita de mecanismul de replicare MongoDB.

SHARDING

Am configurat local un cluster MongoDB sharded folosind Docker Compose, pentru a simula scalarea orizontala (sharding) pe un set mare de date.

Arhitectura:

- **configsvr** = config server (replica set csrs) – stocheaza metadata despre sharding (baze shard-uite, shard key, chunks).
- **shard1** = shard server (replica set rs1) – stocheaza o parte din date.
- **shard2** = shard server (replica set rs2) – stocheaza o parte din date.
- **mongos** = router – punctul de acces la cluster; ruteaza interogările catre shard-uri si face merge la rezultate.

```
C:\FACULTATE\An 3\BD\bdnv-project\sharding>docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
0d3517f798e0	mongo:6.0	"docker-entrypoint.s..."	7 seconds ago	Up 5 seconds	0.0.0.0:27017->27017/tcp	mongos
æefdef926f76	mongo:6.0	"docker-entrypoint.s..."	7 seconds ago	Up 6 seconds	27017/tcp, 0.0.0.0:27020->27020/tcp	shard2
816b34fbfaab	mongo:6.0	"docker-entrypoint.s..."	7 seconds ago	Up 6 seconds	27017/tcp, 0.0.0.0:27018->27018/tcp	shard1
f0f7ee48eceb	mongo:6.0	"docker-entrypoint.s..."	7 seconds ago	Up 6 seconds	27017/tcp, 0.0.0.0:27019->27019/tcp	configsvr

Configurare:

1. Am initializat replica set-urile:
 - **csrs** pe configsvr
 - **rs1** pe shard1
 - **rs2** pe shard2
2. M-am conectat la mongos si am adaugat shard-urile in cluster:
 - **rs1/shard1:27018**
 - **rs2/shard2:27020**

- ```
[direct: mongos] test> sh.status()
shardingVersion
{ _id: 1, clusterId: ObjectId('696a00a3ec96708be2f61eb4') }

shards
[
 {
 _id: 'rs1',
 host: 'rs1/shard1:27018',
 state: 1,
 topologyTime: Timestamp({ t: 1768554762, i: 3 })
 },
 {
 _id: 'rs2',
 host: 'rs2/shard2:27020',
 state: 1,
 topologyTime: Timestamp({ t: 1768554813, i: 3 })
 }
]

```

3. Am activat sharding pe baza de date gestiune\_conferinte si am shard-uit colectia facturi cu shard key:

- **{ id\_eveniment: "hashed" }**

Dataset pentru test:

- Am generat **50 evenimente** (evenimente\_generated)
- Am generat **10.000 facturi** (facturi) care refera aleator un **id\_eveniment**
- Confirmare: **db.facturi.countDocuments() = 10000**
- Distributie pe **shard-uri** (confirmata cu db.facturi.getShardDistribution()):
  - **rs1: 5624 documente (~56%)**
  - **rs2: 4376 documente (~44%)**

o

```
10000
[direct: mongos] gestiune_conferinte> db.facturi.getShardDistribution()
Shard rs1 at rs1/shard1:27018
{
 data: '581KiB',
 docs: 5624,
 chunks: 2,
 'estimated data per chunk': '290KiB',
 'estimated docs per chunk': 2812
}

Shard rs2 at rs2/shard2:27020
{
 data: '452KiB',
 docs: 4376,
 chunks: 2,
 'estimated data per chunk': '226KiB',
 'estimated docs per chunk': 2188
}

Totals
{
 data: '1MiB',
 docs: 10000,
 chunks: 4,
 'Shard rs1': [
 '56.23 % data',
 '56.24 % docs in cluster',
 '105B avg obj size on shard'
],
 'Shard rs2': [
 '43.76 % data',
 '43.76 % docs in cluster',
 '105B avg obj size on shard'
]
}
```

## Test performanta: targeted vs scatter-gather

Am comparat doua tipuri de interogari folosind explain("executionStats")

### 1. Targeted query (foloseste shard key: id\_eveniment)

```
db.facturi.find({ id_eveniment: oneEvent }).explain("executionStats")
```

Rezultat observat:



```

 shardName: 'rs1',
 executionSuccess: true,
 nReturned: 206,
 executionTimeMillis: 1,
 totalKeysExamined: 206,
 totalDocsExamined: 206,
 executionStages: {
 stage: 'FETCH',
 filter: {
 id_eveniment: { '$eq': Object
 },
 nReturned: 206,
 executionTimeMillisEstimate: 0,
 works: 207,
 advanced: 206,
 needTime: 0,
 needYield: 0,
 saveState: 0,
 restoreState: 0,
 isEOF: 1,
 docsExamined: 206,
 alreadyHasObj: 0,
 inputStage: {
 stage: 'IXSCAN',
 nReturned: 206,
 executionTimeMillisEstimate: 0,

```

- foloseste indexul id\_eveniment\_hashed (IXSCAN)
- executionTimeMillis: ~1 ms
- totalDocsExamined: 206 (doar documentele relevante)

## 2. Scatter-gather query (nu foloseste shard key: suma >= 1800)

```
db.facturi.find({ suma: { $gte: 1800 } }).explain("executionStats")
```

Rezultat observat:

```

executionStats: {
 nReturned: 1219,
 executionTimeMillis: 4,
 totalKeysExamined: 0,
 totalDocsExamined: 10000,
 executionStages: {
 stage: 'SHARD_MERGE',
 nReturned: 1219,
 executionTimeMillis: 4,
 totalKeysExamined: 0,
 totalDocsExamined: 10000,
 totalChildMillis: Long('6'),
 shards: [
 {

```

- plan SHARD\_MERGE (query trimis catre toate shard-urile, apoi rezultate combinate)
- COLLSCAN pe fiecare shard
- totalDocsExamined: 10000 (scanare pe tot datasetul)
- executionTimeMillis: ~4 ms

Trade-offs (concluzie):

- Sharding ajuta la scalare orizontala: datele sunt impartite intre shard-uri, ceea ce permite cresterea volumului si a throughput-ului.
- Daca interogările contin shard key-ul (id\_eveniment), clusterul poate face targeted routing (SINGLE\_SHARD), ceea ce este eficient.
- Interogările care nu folosesc shard key-ul devin scatter-gather (SHARD\_MERGE) si pot scana date pe mai multe shard-uri, ceea ce creste costul.
- Sharding adauga complexitate operationala (mongos, config servers, balancing).

Rezumat comenzi:

```

docker compose up -d
docker ps

```

#INIT configsvr:

```

docker exec -it configsvr mongosh --port 27019

```

```

rs.initiate({ _id: "csrs", configsvr: true, members: [{ _id: 0, host:
"configsvr:27019" }] })
exit

```

```

#Init shard1(rs1)

```

```

docker exec -it shard1 mongosh --port 27018
rs.initiate({ _id: "rs1", members: [{ _id: 0, host: "shard1:27018" }] })
exit

#Init shard2(rs2)
docker exec -it shard2 mongosh --port 27020
rs.initiate({ _id: "rs2", members: [{ _id: 0, host: "shard2:27020" }] })
exit

#conectare la mongos, adaugare shard-uri, sharding

docker exec -it mongos mongosh --port 27017 sh.addShard("rs1/shard1:27018")
sh.addShard("rs2/shard2:27020")

sh.enableSharding("gestiune_conferinte")
sh.shardCollection("gestiune_conferinte.facturi", { id_eveniment: "hashed" })

sh.status()

#generare date si verificari
use gestiune_conferinte

db.evenimente_generated.deleteMany({})
db.facturi.deleteMany({})

let ev = []
for (let i = 1; i <= 50; i++) {
 ev.push({ nume: "Eveniment Sharding " + i, data_inceput: new Date(2026,0,1+(i%28)), data_sfarsit: new Date(2026,0,2+(i%28)) })
}
db.evenimente_generated.insertMany(ev)

const eventIds = db.evenimente_generated.find({}, { _id: 1 }).toArray().map(x => x._id)

const N = 10000
const BATCH = 1000
let batch = []

for (let i = 1; i <= N; i++) {

```

```

batch.push({ id_eveniment: evId, suma: Math.floor(Math.random()*2000)+50,
descriere: "Factura #" + i, createdAt: new Date() })
if (batch.length === BATCH) { db.facturi.insertMany(batch); batch = [] }
}
if (batch.length) db.facturi.insertMany(batch)

db.facturi.countDocuments()
db.facturi.getShardDistribution()

#Test performance
const oneEvent = db.evenimente_generated.findOne({}, { _id: 1 })._idLink

db.facturi.find({ id_eveniment: oneEvent }).explain("executionStats")
db.facturi.find({ suma: { $gte: 1800 } }).explain("executionStats")

```

Link video sharding:

 <https://youtu.be/96eSMxxUIoM>

## VERTICAL & HORIZONTAL SCALING

### Vertical scaling

Vertical scaling inseamna cresterea resurselor unui singur server (mai mult CPU, RAM, disk).

- Este simplu de implementat
- Nu necesita modificari de arhitectura
- Are limite fizice (nu poti scala la infinit)

In contextul proiectului, vertical scaling ar inseamna rularea MongoDB pe un server mai puternic pentru a gestiona mai multe date si cereri.

### Horizontal scaling

Horizontal scaling inseamna adaugarea mai multor noduri in sistem.

- In MongoDB se realizeaza prin **replication** si **sharding**
- Permite cresterea volumului de date si a numarului de cereri
- Oferă toleranta la erori (replication) si distributia datelor (sharding)
- Este mai complex de configurat

In acest proiect, horizontal scaling a fost demonstrat prin:

- replica set (pentru disponibilitate)
- sharding (pentru distributia unui dataset mare de facturi)

### Concluzie

Vertical scaling este potrivit pentru aplicatii mici sau medii, cu cerinte reduse. Horizontal scaling este preferabil atunci cand volumul de date si numarul de utilizatori cresc si este

necesara atat performanta, cat si disponibilitatea sistemului.

## Common pitfalls in schema design and CRUD operations

In timpul implementarii bazei de date au fost identificate cateva probleme comune specifice proiectarii NoSQL si utilizarii MongoDB:

- **Over-embedding:** includerea prea multor date intr-un singur document poate duce la documente foarte mari si la operatii de update costisitoare.
- **Lipsa indexurilor:** rularea query-urilor pe colectii mari fara index duce la COLLSCAN si la scaderea performantei.
- **Alegerea gresita a shard key-ului:** un shard key cu cardinalitate mica sau distributie neuniforma poate produce dezechilibru intre shard-uri.
- **Modelarea incorecta a relatiilor many-to-many:** lipsa colectiilor de legatura reduce flexibilitatea si reutilizarea datelor.
- **Update-uri frecvente pe date embedded:** modificarile dese in documente denormalizate pot afecta performanta.
- **Lipsa validarii datelor:** fara reguli de validare, pot aparea date inconsistente sau invalide in colectii.

Aceste capcane au fost luate in considerare in alegerea strategiilor de modelare si scalare utilizate in proiect.

## Trade-offs intre flexibilitate, performanta si consistenta in NoSQL

MongoDB ofera un nivel ridicat de flexibilitate, permitand definirea dinamica a structurii documentelor si combinarea strategiilor de embedding si referencing. Aceasta flexibilitate vine cu trade-off-uri: embedding-ul ofera performanta buna pentru operatii de tip read, dar poate afecta consistenta si costul update-urilor, in timp ce referencing-ul ofera consistenta mai buna si flexibilitate la modificari, dar necesita interogari mai complexe. Alegerea strategiei depinde de tiparele de acces la date si de frecventa operatiilor de citire si scriere.

## Cand MongoDB este mai potrivit decat SQL pentru acest dataset

MongoDB este o alegere potrivita pentru acest proiect deoarece domeniul gestionarii evenimentelor de tip conferinta implica structuri variabile, relatii complexe (1:N si N:N) si

necesitatea combinării datelor normalizate cu date denormalizate pentru performanță. Suportul pentru documente, aggregation pipeline, indexare flexibilă, precum și mecanismele de scalare orizontală (replication și sharding) fac MongoDB mai potrivit decât o bază de date SQL clasică în acest context, unde schema poate evolua și volumul de date poate crește.